

ARM Arithmetic and Logical Instructions

Arithmetic operations, bitwise logic, barrel shifter scaling, and immediate constant generation in AArch64

Topic 5 Outline

Topic Objective

Analysis of AArch64 Instruction Set Architecture specifications for arithmetic computation, bitwise logic manipulation, barrel-shifter operations, and immediate constant handling.

- **Section 1:** AArch64 Arithmetic Instructions and Operand Sizing
- **Section 2:** Logical, Bitwise, and Shift Operations
- **Section 3:** Immediate Values, Constants, and Advanced Synthesis
- **Section 4:** Summary and Worked Self-Test Exercises

Foundations of Data Processing

Role of Data-Processing Instructions

Data-processing instructions form the core computational mechanism of the CPU, transforming data held in general-purpose registers without direct memory access.

- Executes entirely within the CPU core datapath.
- Separates compute instructions from memory load and store operations.
- Provides uniform instruction execution speed and pipeline predictability.

AArch64 Arithmetic Instructions and Operand Sizing

Arithmetic Instructions in AArch64

AArch64 utilizes a three-operand assembly syntax for computation:

- **Syntax Format:** `mnemonic destination, source1, source2`
- Operands target 32-bit registers (`w0–w30`) or 64-bit registers (`x0–x30`).
- Arithmetic instructions execute within the CPU datapath using general-purpose registers.

Core Arithmetic Opcodes

- `add` / `adds`: Addition (with optional flag setting).
- `sub` / `subs`: Subtraction (with optional flag setting).
- `neg`: Negation ($0 - \text{operand}$).

Operand Sizing: W versus X Registers

The register prefix determines whether an operation processes 32-bit or 64-bit data:

32-Bit Operations (W registers)

- Utilizes registers w0 through w30.
- Computes 32-bit results.
- **Zero-Extension:** Automatically clears the upper 32 bits of the parent 64-bit X register.

64-Bit Operations (X registers)

- Utilizes registers x0 through x30.
- Computes full 64-bit doubleword results.
- Native size for memory addresses and 64-bit integer variables.

Executing Basic Addition and Subtraction

Representative examples of arithmetic manipulation in AArch64 assembly:

- `add x0, x1, x2` $\rightarrow x0 \leftarrow x1 + x2$ (64-bit addition)
- `sub x3, x4, x5` $\rightarrow x3 \leftarrow x4 - x5$ (64-bit subtraction)
- `add w0, w1, w2` $\rightarrow w0 \leftarrow w1 + w2$ (32-bit addition, upper 32 bits cleared)

Design Principle

Registers serve as primary operands for all arithmetic transformations; memory contents cannot be modified directly by `add` or `sub` instructions.

The Hardwired Zero Register

AArch64 provides a dedicated register that constantly reads as zero and discards write operations:

- **xzr**: 64-bit zero register.
- **wzr**: 32-bit zero register.

Common Idioms Using Zero

- **Register Copy**: `add x0, x1, xzr` computes $x0 \leftarrow x1 + 0$, copying `x1` into `x0`.
- **Register Negation**: `sub x0, xzr, x1` computes $x0 \leftarrow 0 - x1$, negating the value.
- **Comparison Alias**: Comparing a register against zero uses `subs xzr, x0, #0`.

Section 02

Logical, Bitwise, and Shift Operations

Logical and Bitwise Instructions

Bitwise instructions perform parallel boolean logic across individual bits of a register:

- **and**: Bitwise AND (applied for bit masking and bit clearing).
- **orr**: Bitwise OR (applied for bit setting).
- **eor**: Bitwise Exclusive-OR (applied for bit toggling or register zeroing via `eor x0, x0`).
- **bic**: Bitwise Bit Clear (AND NOT: clears bits in destination specified by mask).
- **mvn**: Bitwise NOT (moves negated source into destination).

Practical Uses of Bitwise Logic

Bitwise instructions provide foundational support for low-level system programming and hardware control:

Bit Masking (`and`)

Extracts specific bit fields by clearing unwanted bits.

- `and x0, x1, 0x0F` isolates the lower nibble of `x1`.

Bit Setting (`orr`)

Forces specific bits to 1 without altering other bit positions.

- `orr x0, x1, 0x80` sets bit 7.

Advanced Bitwise Opcodes: `bic` and `mvn`

Bitwise Clear (`bic`)

The `bic` instruction executes an AND operation between the first source and the bitwise NOT of the second source:

$$\text{destination} \leftarrow \text{source1 AND (NOT source2)}$$

- Used for clearing specific flag bits or status masks.

Bitwise NOT (`mvn`)

The `mvn` (Move Not) instruction inverts all bits of the source operand prior to placement in the destination:

$$\text{destination} \leftarrow \text{NOT source}$$

Register Zeroing Idioms

Clearing registers is a frequent operation in compiled machine code:

- While `mov x0, #0` is valid, compilers frequently emit exclusive-or idioms:

```
eor x0, x0, x0
```

- **Rationale for EOR:**

- XORing any value with itself evaluates to zero ($X \oplus X = 0$).
- Avoids dependency on constant generation pools, executing efficiently in hardware datapath pipelines.

Shift Instructions in AArch64

AArch64 includes dedicated shift instructions for positional bit movement:

- **lsl** (Logical Shift Left): Shifts bits left, filling low-order bits with zeros (multiplies by 2^n).
- **lsr** (Logical Shift Right): Shifts bits right, filling high-order bits with zeros (unsigned division by 2^n).
- **asr** (Arithmetic Shift Right): Shifts bits right while preserving the sign bit (signed division by 2^n).
- **ror** (Rotate Right): Shifts bits right, wrapping shifted-out bits into high-order positions.

The AArch64 Barrel Shifter

Hardware Integration

AArch64 incorporates a hardware **barrel shifter** attached to the second operand input of data-processing instructions, allowing a shift operation to occur within the same clock cycle as computation.

- **Example Syntax:** `add x0, x1, x2, lsl #3`
- **Semantic Evaluation:** Computes $x0 \leftarrow x1 + (x2 \times 8)$ in a single cycle.
- Removes the requirement for standalone shift instructions during array indexing and scaling operations.

Barrel Shifter Scale Variations

The barrel shifter supports multiple modifier types on the final register operand:

- `lsl #imm` (Logical Shift Left by immediate)
- `lsr #imm` (Logical Shift Right by immediate)
- `asr #imm` (Arithmetic Shift Right by immediate)

Compiler Optimization Pattern

When source code multiplies a variable by a constant power of two (e.g., `val * 16`), compilers emit a shifted add or move rather than a multiplication instruction:

```
lsl x0, x1, #4
```

Shift Amount Constraints

Shift operations in AArch64 adhere to strict architectural limits:

- For 32-bit (W) register operations, shift amounts range from 0 to 31.
- For 64-bit (X) register operations, shift amounts range from 0 to 63.
- Shifting by an amount equal to or exceeding the register width produces undefined results or assembler errors.

Sign Maintenance

Logical shifts pad with zeros, whereas arithmetic right shift (asr) replicates the MSB to maintain two's complement sign validity.

Section 03

Immediate Values, Constants, and Advanced Synthesis

Handling Constants (Immediates)

Arithmetic and logical instructions frequently combine registers with constant values:

- **Immediate Operand:** A constant numeric value embedded directly within the instruction word.
- In fixed-width 32-bit architectures like AArch64, instructions cannot contain arbitrary 64-bit constants due to space constraints in the instruction word.
- Hardware mechanisms manage constant generation without requiring full 64-bit memory loads.

Immediate Limits in Arithmetic Instructions

Arithmetic Immediates (add, sub)

Standard add and subtract immediate values are restricted to:

- Unsigned 12-bit integers ranging from 0 to 4095 (0x0xFFF).
- Optional hardware bit-shift modifier of 12 bits (multiplying the immediate by 4096), enabling addition of larger aligned constants.
- Example: `add x0, x1, #1024` is valid.
- Arbitrary 64-bit constants require loading into a temporary register first.

Logical Immediates and Bitmask Encoding

Logical instructions (`and`, `orr`, `eor`) handle constants via a bitmask encoding system:

- Logical immediates are formed by replicated bit patterns across the 32-bit or 64-bit word.
- Encoded using combinations of element size, rotation, and pattern repetition.
- Enables common bitmasks (such as byte flags or power-of-two selectors) to be embedded directly within single-cycle logical instructions.

Loading Large Constants: `movz` and `movk`

When constants exceed immediate limits, AArch64 utilizes explicit move instructions:

`movz` (Move Wide with Zero)

- Moves a 16-bit immediate into a specific 16-bit chunk of the register.
- Clears all other bits in the register to zero.

`movk` (Move Wide with Keep)

- Moves a 16-bit immediate into a specific chunk.
- **Keeps** all other register bits untouched, enabling multi-step constant construction.

Synthesizing Complex Expressions

Compilers combine arithmetic, logical instructions, and barrel shifts to evaluate expressions:

C Statement: `x = (a + (b « 2)) & 0xFF;`

Assembly translation sequence:

- ❶ Scale *b* and add to *a*: `add x0, x1, x2, lsl #2`
 - ❷ Mask lower byte: `and x0, x0, #0xFF`
- The barrel shifter merges the shift and add into a single hardware operation, reducing cycle count and instruction footprint.

Connecting Instructions to Condition Flags

Arithmetic and logical operations can update or test condition flags for control flow:

- **Arithmetic Flag Updates:** Using `adds` or `subs` updates PSTATE NZCV flags based on results.
- **Logical Flag Updates:** Logical instructions like `ands` update flags (setting $Z = 1$ if the result is zero, and N based on the MSB).
- Applied for checking bit masks or testing register zero status without requiring a separate comparison instruction.

Performance Implications in Datapaths

Instruction complexity directly impacts CPI (Cycles Per Instruction):

- **Single-Cycle Instructions:** Basic arithmetic (`add`), bitwise logic (`and`), and barrel-shifted operations execute in a single pipeline cycle.
- **Multi-Step Sequences:** Loading large 64-bit constants requires multiple instructions (`movz` followed by `movk`), increasing instruction count.
- ISA design maps frequent operations to single-cycle hardware pathways.

Summary & Self-Test

Topic 5 Comprehensive Summary

- **Arithmetic Instructions:** Execute addition, subtraction, and negation across 32-bit (W) and 64-bit (X) registers.
- **Logical Instructions:** Execute parallel bitwise boolean operations (and, orr, eor, bic, mvn) for masking and setting bits.
- **Shift Operations:** Support scaling via lsl, lsr, asr, and ror, with hardware barrel-shifter integration.
- **Constant Generation:** Managed through 12-bit arithmetic immediates, logical bitmasks, and wide moves (movz/movk).

Self-Test: Questions 1 & 2

Question 1: Operand Sizing

What is the structural difference between operating on 32-bit W registers versus 64-bit X registers in AArch64, and what happens to the upper bits of a register when a 32-bit instruction executes?

Question 2: Bitwise Logic

How would you use AArch64 logical instructions to clear bit 4 of register x0 while leaving all other bits completely unaffected?

Self-Test: Solutions 1 & 2

Solution 1

W registers access the lower 32 bits of the 64-bit X registers. When a 32-bit instruction executes, AArch64 automatically zero-extends the result by clearing the upper 32 bits of the parent register to zero.

Solution 2

Bit 4 can be cleared using a bit clear instruction with a mask containing a 1 at bit 4: `bic x0, x0, #(1 << 4)` (or an explicit immediate mask).

Self-Test: Questions 3 & 4

Question 3: The Barrel Shifter

What is the architectural advantage of the AArch64 built-in barrel shifter during data-processing instructions?

Question 4: Constant Loading

Why can't an AArch64 arithmetic instruction contain an arbitrary 64-bit constant directly, and how do compilers solve this limitation?

Self-Test: Solutions 3 & 4

Solution 3

The barrel shifter allows a source operand to be shifted or scaled as part of the arithmetic or logical instruction in a single clock cycle, avoiding extra standalone shift instructions.

Solution 4

Because instruction words are fixed at 32 bits, storage space is insufficient for both opcodes, register identifiers, and a 64-bit constant. Compilers resolve this using 12-bit immediates, logical bitmasks, or wide move sequences (`movz` / `movk`).

Review: Instruction Summary Table

Instruction Category	Common Opcodes	Primary Architectural Purpose
Arithmetic	add, sub, neg	Integer computation across W/X registers
Logical / Bitwise	and, orr, eor, bic	Masking, setting, clearing, and toggling bits
Shifts	lsl, lsr, asr, ror	Multiplying/dividing by powers of two
Constants	movz, movk	Constructing wide constants across register chunks

Review Exercises & Practice Problems

Self-Directed Practice

Lab exercises for Topic 5 concept reinforcement:

- Write an AArch64 assembly snippet to extract bits 12 through 15 from register `x1` into `x0`.
- Apply a barrel-shifted add instruction to compute $x5 = x6 + (x7 \times 32)$ in a single cycle.
- Explain why `eor x1, x1, x1` is preferred by compilers over `mov x1, #0`.

Advanced Laboratory Preparation

Next Lab Preview

Upcoming laboratory session transition from register-level arithmetic to memory load and store mechanics:

- Inspecting register dumps in QEMU during arithmetic transformations.
- Profiling execution time differences between barrel-shifted operands and standalone shift instructions.

Wrap-Up and Next Steps

Moving Forward to Topic 6

Session transition overview:

- **Memory Access Instructions:** Data loading and storing using `ldr` and `str`.
- **Advanced Addressing Modes:** Post-index, pre-index, and register-offset memory mapping in AArch64.

End of Topic 5 — Questions & Discussion